TP 9 : Des machines de Turing

Ce TP est à réaliser en `C`.

Préliminaires

Les machines de Turing sont un modèle théorique construit pour représenter des ordinateurs. Dans ce sujet, on s'intéresse à leur simulation et non aux nombreuses propriétés théoriques qu'elles possèdent.

L'une des grandes contributions scientifiques d'Alan Turing a été l'introduction d'une formalisation du calcul mécanique aujourd'hui appelée machine de Turing. Une machine de Turing peut être vue comme un automate fini équipé d'une bande infinie sur laquelle se déplace une tête qui peut lire et écrire des 0 ou des 1. La notion de machine de Turing est utile pour raisonner sur l'ensemble des fonctions calculables par ordinateur : informellement, tout ce qui pourrait calculer un ordinateur disposant d'une quantité infinie de mémoire peut être calculé par une machine de Turing, et inversement. Le but du sujet est de simuler l'exécution de machines de Turing afin de trouver des machines qui s'arrêtent soit après un nombre maximum d'étapes, soit en laissant un maximum de 1 écrits sur la bande. Ce problème, appelé jeu du castor affairé (busy beaver dans la langue de Shakespeare), est lié à des notions importantes en informatique théorique.

Définitions

Une **machine de Turing** (ou une machine dans la suite du sujet) est une paire (Q, δ) :

- Q est un ensemble fini d'états, qui sera toujours dans ce sujet $\{0, \dots, n-1\}$ pour un certain $n > 0$,
- $\delta : Q \times \{0,1\} \to (Q \cup \{\perp\}) \times \{0,1\} \times \{-1,1\}$ est la fonction de transition (ou la table des transitions).

0 est l'état initial, et $\perp$ est un état spécial qui sera appelé *l'état terminal*; quand l'exécution d'une machine de Turing atteint l'état terminal, elle s'arrête.

Une **transition** est une paire $(q, x) \in Q \times \{0,1\}$. Une transition (q, x) est dite *terminale* si $\delta(q, x) = (\perp, y, d)$ pour un $y \in \{0,1\}$ et $d \in \{-1,1\}$.

Informellement, $\delta(q, x)$ décrit ce que fait la machine quand elle se trouve en état q, et que le symbole courant en face de la tête de lecture est x. Si $\delta(q, x) = (q', y, d)$, alors la machine va (dans cet ordre) :

1. écrire y à la position courante,
2. déplacer la tête de d cases (si $d = 1$, alors la tête se déplace d'une case vers la droite, et si $d = -1$, la tête se déplace d'une case vers la gauche), et
3. passer dans l'état q'.

Ce mécanisme sera décrit clairement dans la suite de ce sujet. Tout au long du sujet, on utilisera $\leftarrow$ comme un synonyme de -1, et $\rightarrow$ comme un synonyme de 1 lors des déplacements de la tête.

À l'instar d'un automate fini, une machine de Turing peut être représentée par un graphe orienté, où les sommets sont les états et les arcs représentent les transitions. Si $\delta(q, x) = (q', y, d)$, alors il y aura un arc entre q et q', et cet arc sera étiqueté $x : y, d$.

On représente dans ce sujet, une machine par son nombre d'états, numéroté de 0 à n. L'état numéro n sera l'état $\perp$. Pour représenter la fonction de transition, on utilisera un tableau de dimension deux contenant des triplets d'entiers. La case à la position i, j représentera le résultat de la fonction $\delta(i, j)$.

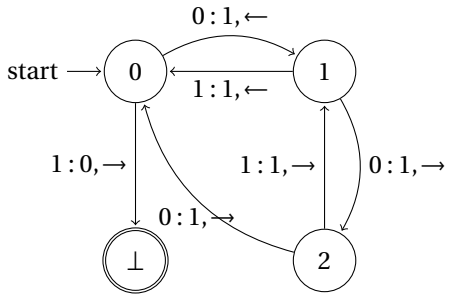
```
struct triplet
{
  int etat;
  int lettre;
  int mouvement;
};

struct machine_t
{
  int nb_etats;
  struct triplet** delta;
};

typedef struct machine_t machine;
```

$q \setminus x$	0	1
0	$(1, 1, \leftarrow)$	$(\perp, 0, \rightarrow)$
1	$(2, 1, \rightarrow)$	$(0, 1, \leftarrow)$
2	$(0, 1, \rightarrow)$	$(1, 1, \rightarrow)$

(a) Tableau de δ



(b) Représentation en automate

FIGURE 1 – Machine de Turing : tableau de δ et automate

Question 1 : Écrire une fonction `machine initialiser_machine(int n)` qui, étant donné un nombre d'états, renvoie une machine dont toutes les transitions envoient dans l'état 0, écrivent un 0 et se déplacent vers la droite.

Un code générant la machine de la figure 2 vous est fourni en fin de sujet.

Question 2 : Écrire une fonction `int nombre_terminales(machine M)` qui, étant donnée une machine de Turing, renvoie le nombre de transitions envoyant sur l'état $\perp$.

Exécution d'une machine de Turing

Pour exécuter une machine de Turing, on utilise une tableau infini des deux côtés dont chaque case contient un 0 où un 1. Initialement, le tableau est rempli de 0. À chaque itération, la machine lit la case en face de sa tête de lecture et effectue la transition correspondante. Elle change alors d'état, écrit dans la case devant sa tête de lecture puis se déplace. Un exemple d'exécution est proposé dans la figure 2.

Question 3 : Combien d'étapes faut-il à la machine de la figure 1 pour s'arrêter ? Quel sera alors le nombre de 1 sur le tableau ?

Dans ce sujet, on ne dépassera jamais 250 étapes, et on ne dépassera donc jamais un tableau de 501 cases (250 cases vers la droite et autant vers la gauche). On utilisera donc un deux tableaux de taille finie.

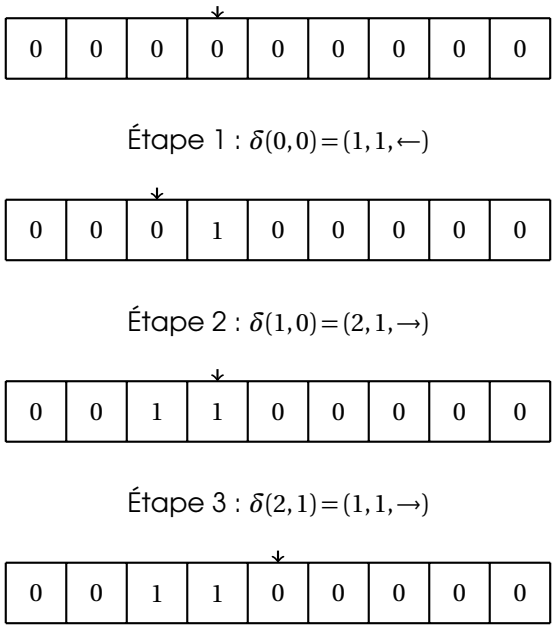


FIGURE 2 – Début de l'exécution de la machine de la figure 1.

```
typedef struct configuration_t
{
  int* tableau_positif;
  int* tableau_negatif;
  int position;
  int etat;
};

typedef struct configuration_t configuration;
```

L'attribut `tableau_positif` contient les valeurs des cases de 0 à 250 tandis que `taleau_negatif` contient les case de -1 à -250. Enfin, `position` représente la position de la tête de lecture tandis que `etat` représente l'état actuel de la machine de Turing dans sont automate.

Question 4 : Écrire une fonction `configuration initialiser()` qui crée une instance de `configuration` avant la première étape.

Question 5 : Écrire une fonction `int trouver_case(configuration c, int indice)` qui, étant donné une instance de `configuration` et l'indice d'une case, renvoie la valeur stockée dans celle-ci. On fera attention à bien traiter les cas négatifs.

Question 6 : Écrire une fonction `void etape_suivante(machine M, configuration* c)` qui, étant donné une configuration d'une machine de Turing, la modifie pour représenter le calcul d'une étape.

Question 7 : Écrire une fonction `bool termine_partiel(machine M, int k)` qui, étant donnée une machine, teste si celle-ci s'arrête en strictement moins de k étapes.

Détection de non terminaison

Pour répondre au problème du castor affairé, il faut savoir détecter si une machine termine. Il s'agit d'un problème indécidable mais on peut détecter dans certains cas qu'une machine ne s'arrête pas.

Ensemble bouclant

Dans certains cas, on peut savoir qu'une exécution ne s'arrêtera pas, sans pour autant comprendre ce que fait la machine : par exemple, lorsque la machine ne possède pas de transition finale. Un sous-ensemble d'états $Q' \subseteq Q$ est bouclant si pour tout $q \in Q'$ et pour tout $x \in 0, 1, \delta(q, x) = (q', y, d)$ avec $q' \in Q'$.

Si l'exécution d'une machine arrive dans une état $q \in Q'$ pour un sous-ensemble bouclant Q', alors l'exécution ne termine pas. En effet, les états ultérieurs de l'exécution resteront dans Q', sans possibilité d'arriver sur $\perp$. Étant donné une machine M , on note $L(M)$ l'union de tous les ensembles bouclants de M. On le représentera par un tableau de booléens de taille n indiquant pour chaque état s'il appartient à un ensemble bouclant.

Question 8 : Écrire une fonction `bool* ensemble_bouclant(machine M)` qui, étant donnée une machine de Turing, renvoie $L(M)$.

Question 9 : Écrire une fonction `bool passe_test(machine M)` qui, étant donnée une machine de Turing, renvoie `true` si et seulement si la machine fournie passe le test proposé. Que peut-on dire des cas où elle renvoie `false`?

Configurations bouclantes

Dans la majorité des cas restants, il faut comprendre ce que fait la machine pour pouvoir détecter que son exécution ne termine pas. Pour faire cela, il faut regarder la suite des Configurations dans l'exécution, et détecter qu'un schéma va se répéter. Par exemple, s'il existe $i < j$ tels que $C_i = C_j$ dans une exécution $E = (C_0, C_1, \ldots)$, alors cette exécution sera infinie. On se propose de détecter un cas plus général : des exécutions qui vont boucler en répétant le même schéma, mais en se décalant vers la droite.

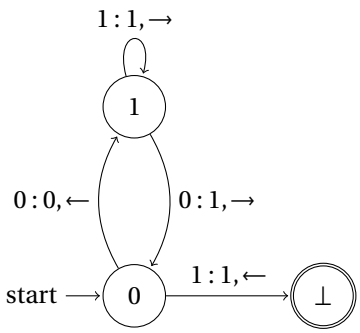


FIGURE 3 – Une machine de Turing qui boucle à droite

Question 10 : En simulant l'exécution de la machine de la figure 3, montrer qu'elle boucle bien vers la droite.

Question 11 : Proposer une machine dont l'exécution est infinie mais qui ne boucle ni à droite ni à gauche.

Question 12 : Écrire une fonction `bool boucle_a_droite(machine M, int k)` qui, étant donnée une machine de Turing, teste si celle-ci boucle à droite en strictement moins de k étapes. On renverra `true` dans le cas où elle s'arrête.

```
machine exemple_machine()
{
  machine M = initialiser_machine(3); // 3 etats sans compter le terminal
  M.delta[0][0] = (struct triplet){1, 1, -1}; // delta(0, 0) = (1, 1, -1)
  M.delta[0][1] = (struct triplet){3, 0, 1};  // delta(0, 1) = (bot, 0, 1)
  M.delta[1][0] = (struct triplet){2, 1, 1};  // delta(1, 0) = (2, 1, 1)
  M.delta[1][1] = (struct triplet){0, 1, -1}; // delta(1, 1) = (0, 1, -1)
  M.delta[2][0] = (struct triplet){0, 1, 1};  // delta(2, 0) = (0, 1, 1)
  M.delta[2][1] = (struct triplet){1, 1, 1};  // delta(2, 1) = (1, 1, 1)
  return M;
}
```